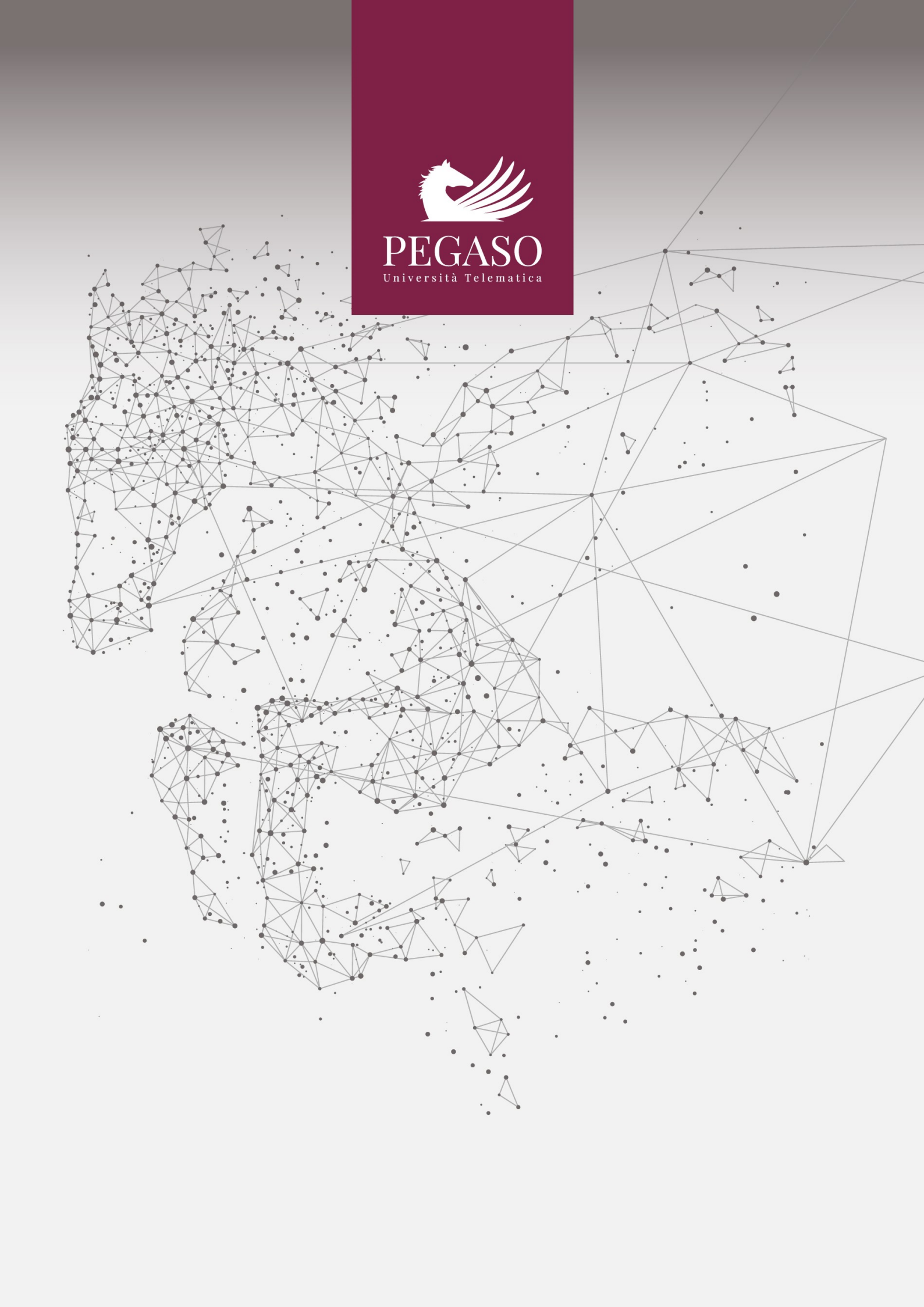

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ALBERO ROSSO-NERO 3
2. ELIMINAZIONE IN ALBERO ROSSO-NERO 4
3. CANCELLAZIONE - ESEMPIO 10
4. IMPLEMENTAZIONE 12
BIBLIOGRAFIA E SITOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Albero rosso-nero

Si dice che un albero binario di ricerca è **rosso-nero** se:

- Ogni nodo è colorato di rosso o di nero
- Le chiavi vengono mantenute solo nei nodi interni dell'albero
- Le foglie sono costituite da nodi speciali Nil: tali nodi sono detti "nodi sentinella" ed il loro contenuto è di fatto "irrilevante". Di fatto, quando in un albero rosso-nero un nodo ha come figli nodi NIL, quel nodo sarebbe una foglia dell'albero binario di ricerca corrispondente
- Vengono rispettati i seguenti vincoli:
  - ✓ La radice è nera;
  - ✓ Tutte le foglie sono nere;
  - ✓ Entrambi i figli di un nodo rosso sono neri;
  - ✓ Ogni cammino semplice da un nodo ad una delle foglie contenute nel suo sottoalbero ha lo stesso numero di nodi neri.

Dalle regole di cui sopra ricaviamo che non possono esserci 2 nodi rossi adiacenti, anche se più nodi neri possono esserlo.

Quando dunque i vincoli rosso-nero vengono violati si può agire tentando di modificare i colori nella zona della violazione oppure, se questo non porta ad una soluzione, operando dei ribilanciamenti dell'albero tramite rotazioni (destra e sinistra).

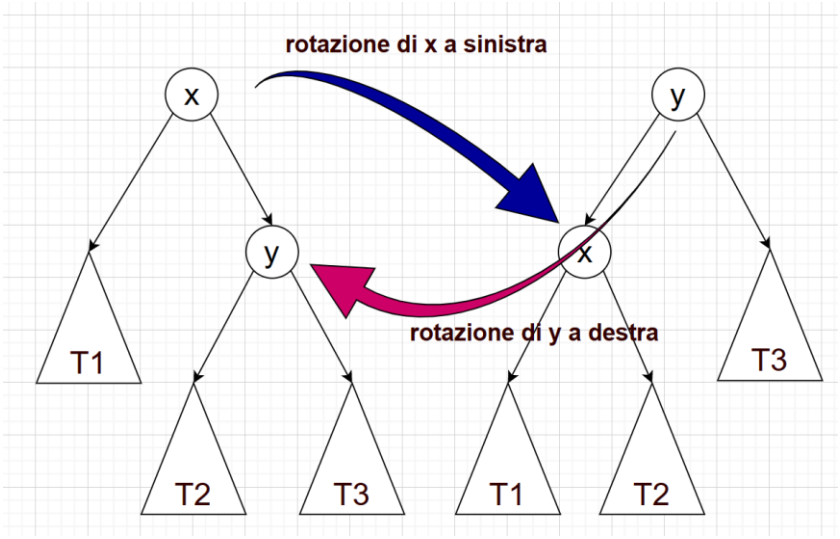


Figura 1: rotazioni

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Eliminazione in albero rosso-nero

L'algoritmo di cancellazione per un albero rosso-nero è costruito sull'algoritmo di cancellazione per alberi binari di ricerca: dopo la cancellazione si deve decidere se è necessario ribilanciare o meno.

Tuttavia, le operazioni di ripristino del bilanciamento sono necessarie solo quando il nodo cancellato è nero, infatti:

- Se il nodo cancellato è rosso:
  - ✓ L'altezza nera invariata;
  - ✓ Non sono stati creati nodi rossi consecutivi;
  - ✓ La radice resta nera.
- Se il nodo cancellato è nero:
  - ✓ Possiamo violare il vincolo 1 secondo cui la radice non può essere un nodo rosso;
  - ✓ Possiamo violare il vincolo 3, cioè se il padre e uno dei figli del nodo cancellato erano rossi;
  - ✓ Abbiamo violato il vincolo 4, cioè altezza nera è cambiata.

Consideriamo lo schema di partenza:

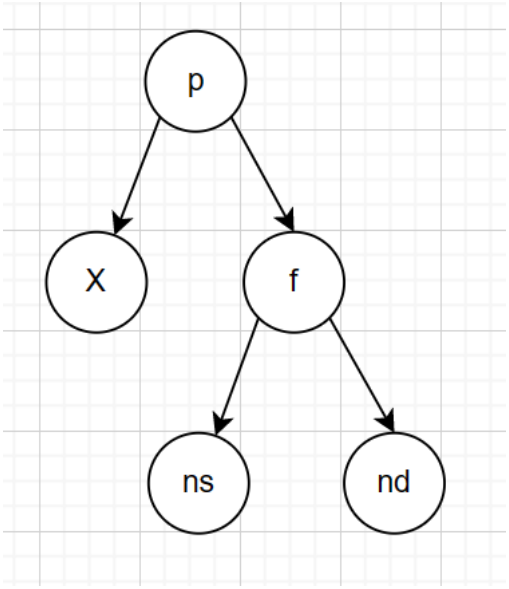


Figura 2: Eliminazione: schema base

- X è il nodo da eliminare
- p è il padre
- f è il figlio
- ns è nipote sinistro
- nd è il nipote destro

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il primo caso si verifica quando il fratello è di colore rosso:

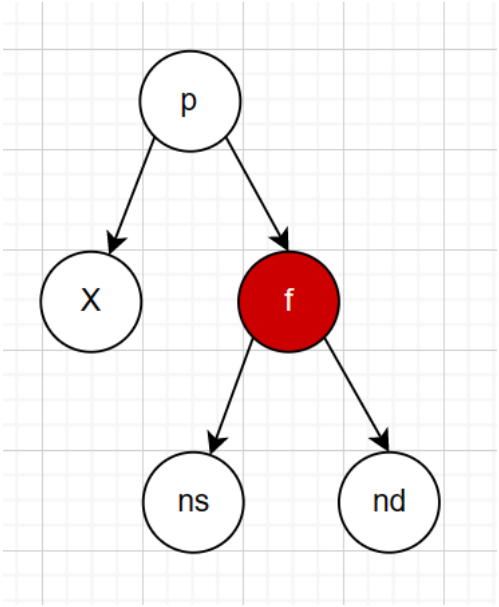


Figura 3: Eliminazione: caso1

I passi da seguire sono i seguenti:

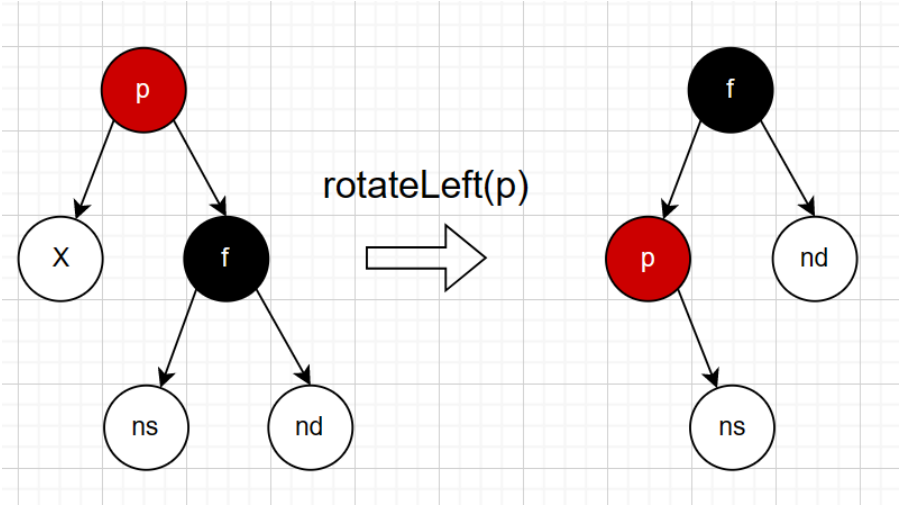


Figura 4: Eliminazione: caso1

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il secondo caso si verifica quando il fratello è di colore nero e lo sono anche i 2 nipoti:

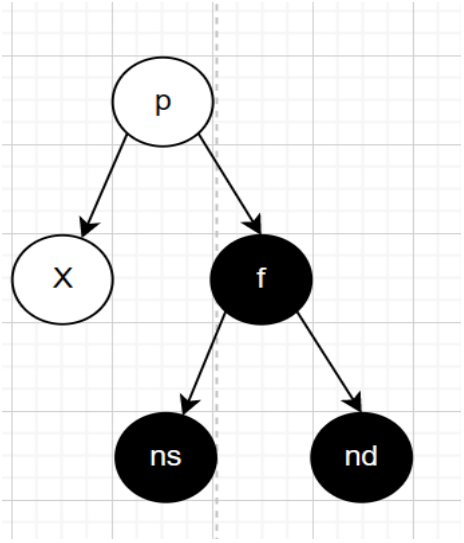


Figura 5: Eliminazione: caso2

I passi da seguire sono i seguenti:

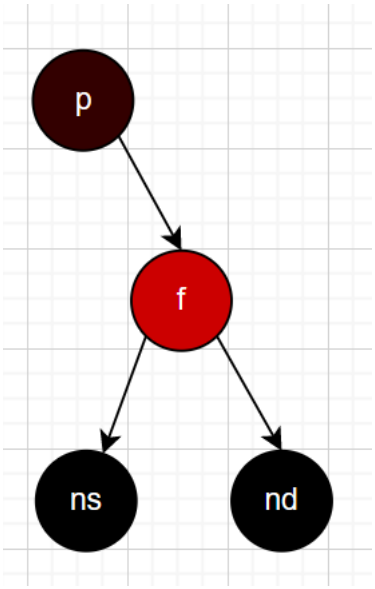


Figura 6: Eliminazione: caso2

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il terzo caso si verifica quando il fratello è di colore nero ed il nipote sinistro è rosso mentre quello destro è nero:

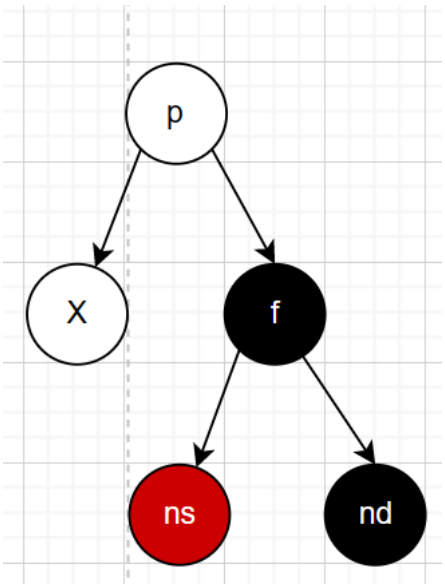


Figura 7: Eliminazione: caso3

I passi da seguire sono i seguenti:

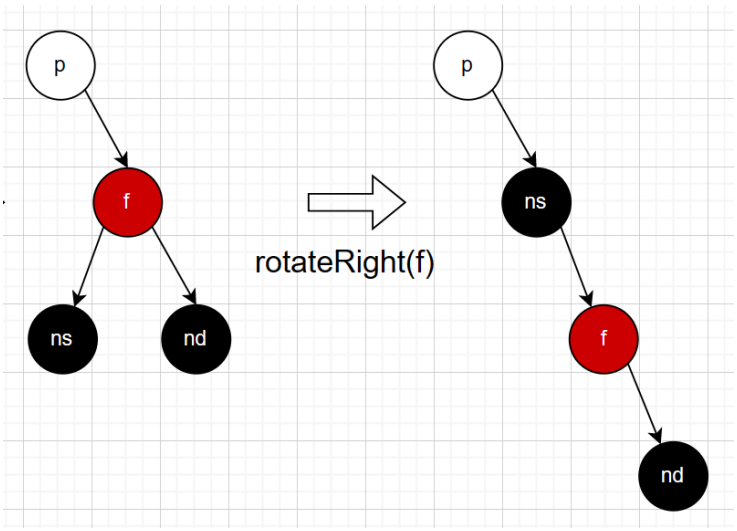


Figura 8: Eliminazione: caso3

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il quarto caso si verifica quando il fratello è di colore nero ed il destro è rosso:

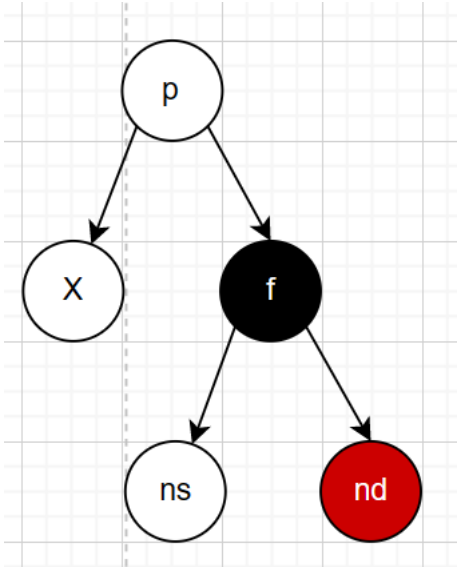


Figura 9: Eliminazione: caso4

I passi da seguire sono i seguenti:

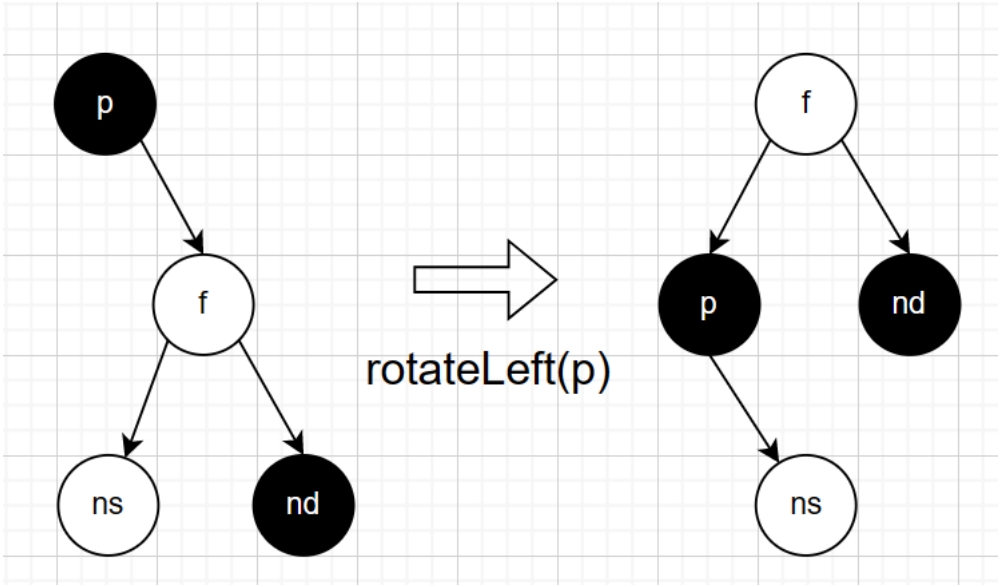


Figura 10: Eliminazione: caso4

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Complessivamente potremmo sintetizzare il tutto con il seguente pseudocodice:

```
while t != T and t.color = BLACK
      Tree p = t.parent                                    % padre
      if t = p.left then
            Tree f = p.right                               % fratello
            Tree ns = f.left                               % nipote sinistro
            Tree nd = f.right                              % nipote destro
            if f.color == RED then                         % (1)
                  p.color = RED
                  f.color = BLACK
                  rotateLeft(p)
                  % t viene lasciato inalterato
                  % quindi si ricade nei casi (2),(3),(4)
            else
                  if ns.color == nd.color == BLACK then              % (2)
                        f.color = RED
                        t = p
                  else if ns.color == RED and nd.color == BLACK then % (3)
                        ns.color = BLACK
                        f.color = RED
                        rotateRight(f)
                        % t viene lasciato inalterato
                        % quindi si ricade nel caso (4)
                  else if nd.color == RED then                       % (4)
                        f.color = p.color
                        p.color = BLACK
                        nd.color = BLACK
                        rotateLeft(p)
                        t = T
      else
            % Casi (5)-(8) speculari a (1)-(4)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Cancellazione - esempio

Consideriamo il seguente esempio:

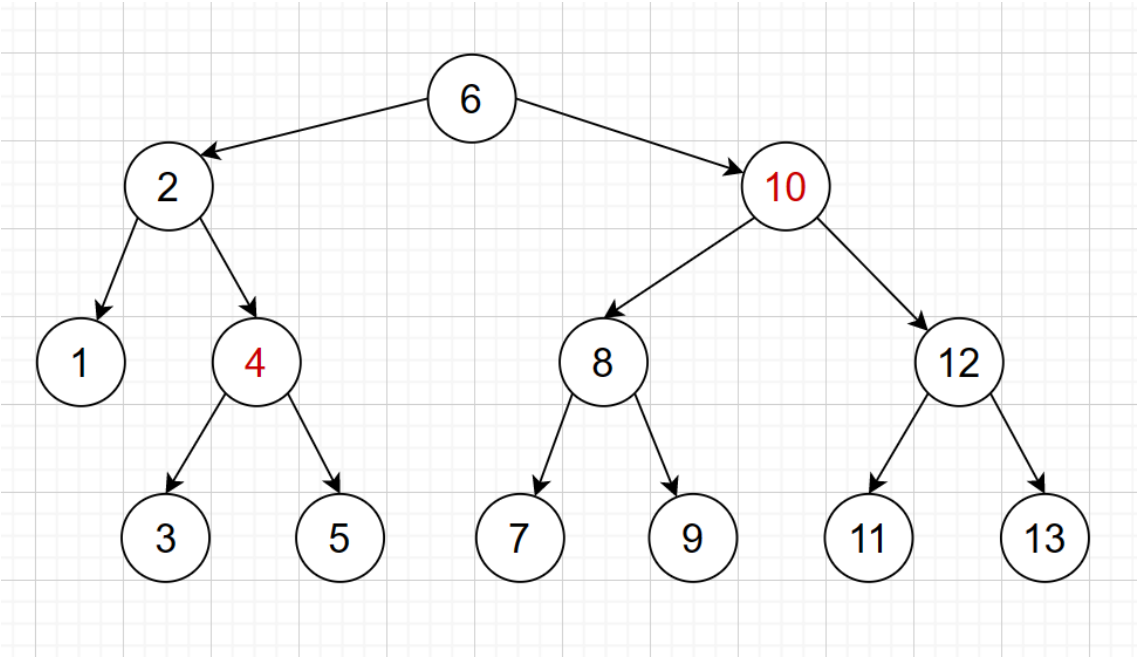


Figura 11: Eliminazione nodo 3

In questo caso ci troviamo nello scenario del caso 2 (fratello nero e nipoti neri) pertanto lo schema finale sarà:

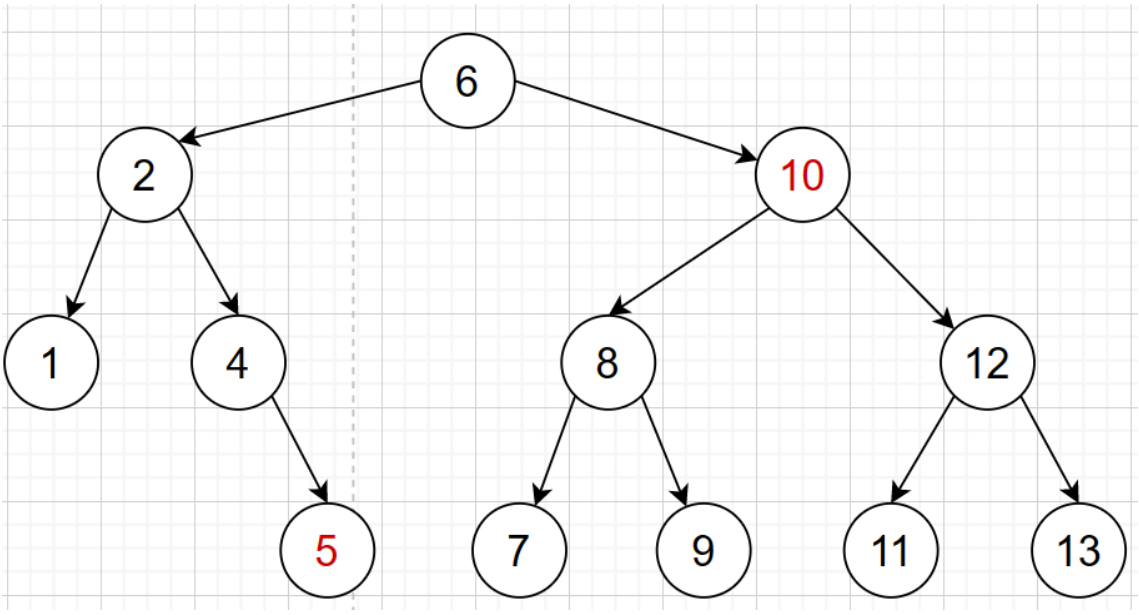


Figura 12: Eliminazione nodo 3

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Consideriamo ora il seguente schema:

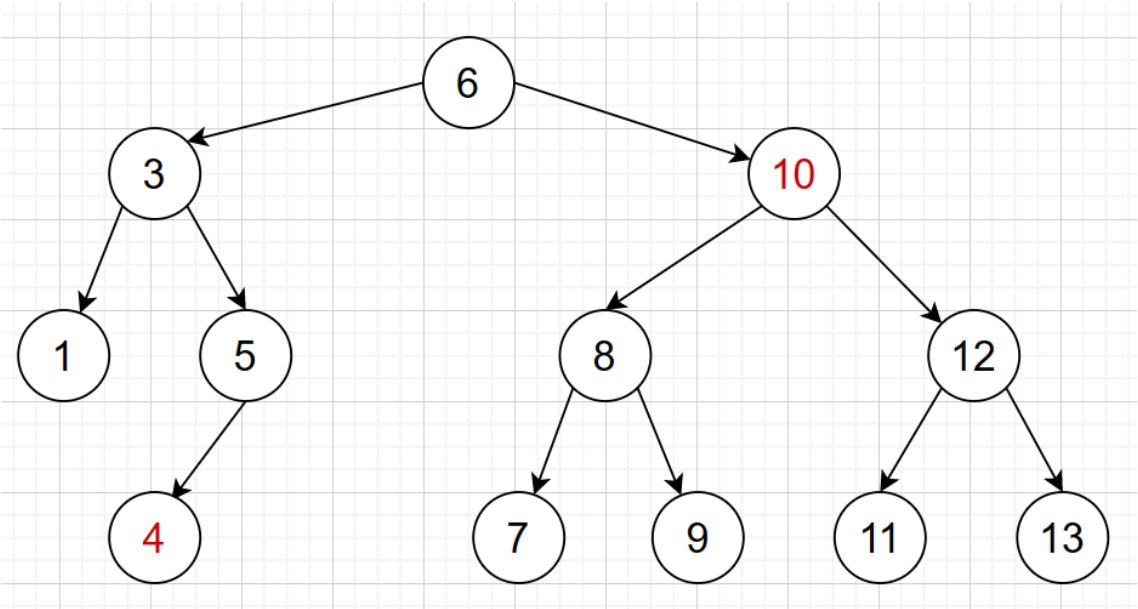


Figura 13: Eliminazione nodo 1

Ci troviamo nel caso 3 (fratello nero, nipote sinistro rosso e destro nero) pertanto lo schema finale sarà:

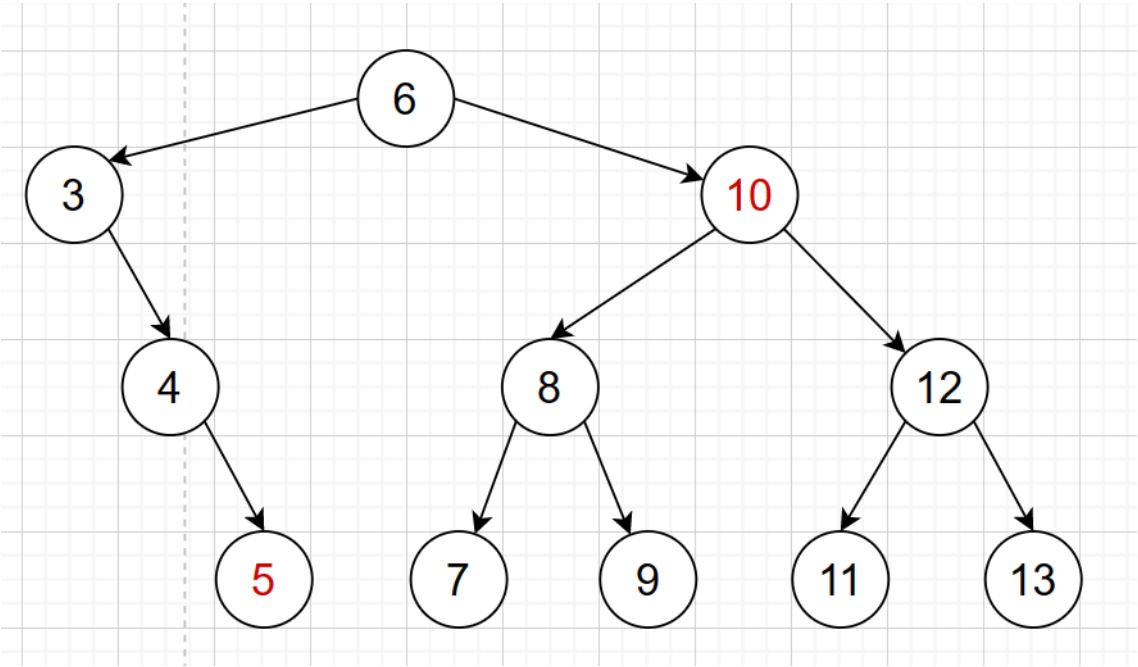


Figura 14: Eliminazione nodo 1

In termini di complessità, è possibile visitare al massimo un numero $O(\log_2 n)$ di casi, ognuno dei quali è gestito in tempo $O(1)$.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Implementazione

Di seguito l'implementazione dell'albero rosso-nero in Python:

```
class Node:
  def __init__(self, value, color='red'):
    self.value = value
    self.color = color
    self.left = None
    self.right = None
    self.parent = None

class RedBlackTree:
  def __init__(self):
    self.root = None

  def insert(self, value):
    new_node = Node(value)
    if self.root is None:
        self.root = new_node
        new_node.color = 'black'
        return
    parent, current = None, self.root
    while current:
        parent = current
        if value < current.value:
            current = current.left
        else:
            current = current.right
    new_node.parent = parent
    if value < parent.value:
        parent.left = new_node
    else:
        parent.right = new_node
    self._repair_tree(new_node)

  def _repair_tree(self, node):
    if node.parent is None:
        node.color = 'black'
        return
    if node.parent.color == 'black':
        return
    uncle = self._get_uncle(node)
    grand_parent = self._get_grand_parent(node)

    if uncle and uncle.color == 'red':
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      node.parent.color = 'black'
      uncle.color = 'black'
      grand_parent.color = 'red'
      self._repair_tree(grand_parent)
    else:
      if node == node.parent.right and node.parent == grand_parent.left:
        self._rotate_left(node.parent)
        node = node.left
      elif node == node.parent.left and node.parent ==
grand_parent.right:
        self._rotate_right(node.parent)
        node = node.right
      grand_parent = self._get_grand_parent(node)
      node.parent.color = 'black'
      grand_parent.color = 'red'
      if node == node.parent.left:
        self._rotate_right(grand_parent)
      else:
        self._rotate_left(grand_parent)

  def _get_uncle(self, node):
    grand_parent = self._get_grand_parent(node)
    if grand_parent is None:
        return None
    if node.parent == grand_parent.left:
        return grand_parent.right
    else:
        return grand_parent.left

  def _get_grand_parent(self, node):
    if node.parent is None or node.parent.parent is None:
      return None
    return node.parent.parent

  def _rotate_left(self, node):
    right = node.right
    node.right = right.left
    if right.left:
        right.left.parent = node
    right.parent = node.parent
    if node.parent is None:
        self.root = right
    elif node == node.parent.left:
        node.parent.left = right
    else:
        node.parent.right = right
    right.left = node
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    node.parent = right

  def _rotate_right(self, node):
    left = node.left
    node.left = left.right
    if left.right:
        left.right.parent = node
    left.parent = node.parent
    if node.parent is None:
        self.root = left
    elif node == node.parent.right:
        node.parent.right = left
    else:
        node.parent.left = left
    left.right = node
    node.parent = left

  def _fix_insert(self, node):
    while node.parent and node.parent.color == 'red':
      if node.parent == node.parent.parent.left:
        uncle = node.parent.parent.right
        if uncle and uncle.color == 'red':
          node.parent.color = 'black'
          uncle.color = 'black'
          node.parent.parent.color = 'red'
          node = node.parent.parent
        else:
          if node == node.parent.right:
            node = node.parent
            self._rotate_left(node)
          node.parent.color = 'black'
          node.parent.parent.color = 'red'
          self._rotate_right(node.parent.parent)
      else:
        uncle = node.parent.parent.left
        if uncle and uncle.color == 'red':
          node.parent.color = 'black'
          uncle.color = 'black'
          node.parent.parent.color = 'red'
          node = node.parent.parent
        else:
          if node == node.parent.left:
            node = node.parent
            self._rotate_right(node)
          node.parent.color = 'black'
          node.parent.parent.color = 'red'
          self._rotate_left(node.parent.parent)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    self.root.color = 'black'

def print_tree_markdown(node, level=0):
    if node is None:
        return
    print_tree_markdown(node.right, level + 1)
    print('    ' * level + f'- {node.value} ({node.color})')
    print_tree_markdown(node.left, level + 1)

def print_red_black_tree_markdown(tree):
    print_tree_markdown(tree.root)

tree = RedBlackTree()
tree.insert(10)
tree.insert(20)
tree.insert(30)
tree.insert(40)
tree.insert(50)
tree.insert(60)
print_red_black_tree_markdown(tree)
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione;
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione;
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley;
- Sedgewick: Algoritmi in C, Pearson, 2015;
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).